# PROGRAMA DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

## Trabalho Prático 2

### Desenvolvimento de uma Biblioteca Modular em C

*Makefile, bibliotecas estática e dinâmica, testes e Git*

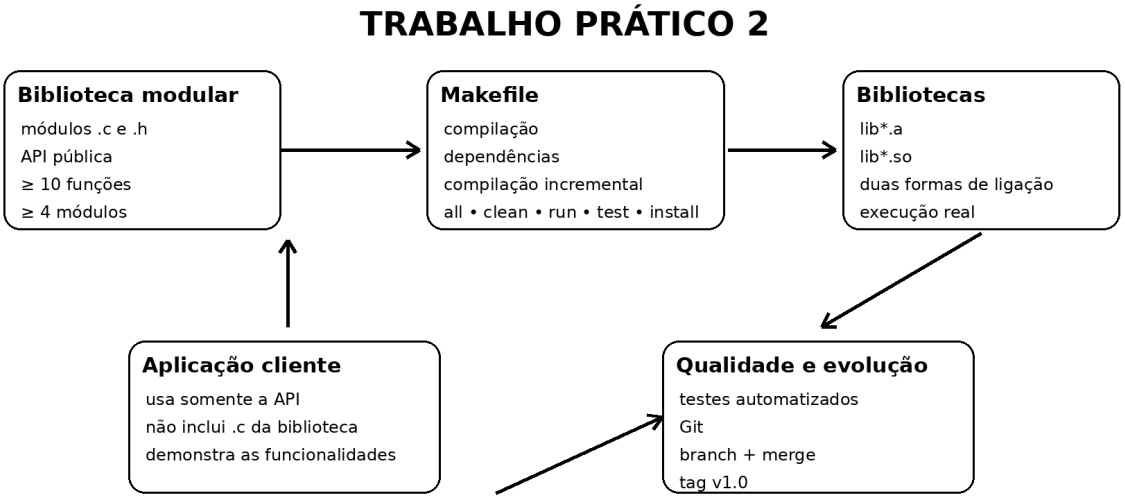



*Figura 1. Visão geral do projeto que deverá ser desenvolvido.*

## 1. Objetivo

O objetivo deste trabalho é projetar, implementar, testar e documentar uma biblioteca reutilizável em linguagem C. O projeto deverá ser organizado em módulos, disponibilizar uma interface pública, ser compilável automaticamente com Make, gerar bibliotecas estática e dinâmica, possuir testes automatizados e utilizar Git para registrar sua evolução.

## 2. O que o grupo deverá construir

Cada grupo deverá escolher um domínio para a biblioteca, sujeito à aprovação do professor. Exemplos incluem matrizes, estatística, estruturas de dados, processamento de strings ou outro tema adequado.

A biblioteca deverá ser utilizada por uma aplicação cliente independente. A aplicação deverá utilizar somente a interface pública da biblioteca, definida nos arquivos .h, sem incluir os arquivos .c da biblioteca ou acessar diretamente seus detalhes internos.

O mesmo projeto deverá permitir a compilação da aplicação utilizando uma biblioteca estática e uma biblioteca dinâmica. O projeto também deverá possuir testes automatizados e um histórico de desenvolvimento em Git.

# 3. Requisitos Funcionais

## 3.1 Biblioteca e modularização

- A biblioteca deverá possuir pelo menos 10 funções públicas.
- As funções públicas deverão estar distribuídas em pelo menos 4 módulos funcionalmente distintos.
- Um módulo será considerado válido quando possuir uma responsabilidade funcional própria e coerente dentro do domínio escolhido.
- A simples divisão artificial de funções em arquivos diferentes não será considerada modularização adequada.
- Cada módulo deverá possuir, quando aplicável, seu arquivo .c de implementação e seu arquivo .h de interface.
- Os arquivos .h deverão apresentar a interface pública da biblioteca.
- Detalhes internos da implementação deverão permanecer nos arquivos .c sempre que possível.
- Cada função pública deverá ser documentada com sua finalidade, parâmetros, retorno e condições de erro, quando aplicáveis.
- Todas as funções públicas deverão ser utilizadas pela aplicação cliente ou pelos testes.
- A biblioteca deverá tratar situações de erro relevantes ao domínio escolhido de forma documentada e consistente.

## 3.2 Interface pública

- A aplicação cliente deverá incluir apenas os arquivos .h necessários para utilizar a biblioteca.
- A aplicação cliente não poderá incluir arquivos .c da biblioteca.
- A aplicação cliente não poderá depender de variáveis ou estruturas internas que não façam parte da API pública.
- A organização deverá permitir modificar a implementação interna de um módulo sem alterações desnecessárias na aplicação cliente.

## 3.3 Biblioteca estática e biblioteca dinâmica

- O projeto deverá gerar uma biblioteca estática no formato .a.
- O projeto deverá gerar uma biblioteca dinâmica no formato .so.
- A aplicação cliente deverá ser compilada e executada utilizando a biblioteca estática.
- A aplicação cliente deverá ser compilada e executada utilizando a biblioteca dinâmica.
- A biblioteca dinâmica deverá ser efetivamente utilizada durante a execução.
- O relatório deverá explicar as diferenças entre ligação estática e ligação dinâmica.

## 3.4 Makefile

O projeto deverá possuir um Makefile com, no mínimo, os seguintes alvos:

```
all       Compila a biblioteca e a aplicação.

clean     Remove os artefatos gerados pela compilação.

run       Compila quando necessário e executa a aplicação.

test      Compila e executa automaticamente os testes.

install   Instala a biblioteca e os arquivos de interface.
```

O Makefile deverá representar corretamente as dependências entre os arquivos. Uma alteração em um módulo não deverá provocar a recompilação desnecessária de arquivos que não dependem dele.

### 3.5 Testes automatizados

- O projeto deverá possuir testes automatizados para todas as funções públicas.
- Deverão existir testes para situações normais de uso.
- Deverão existir testes para valores limite ou entradas inválidas, quando aplicáveis.
- Cada módulo deverá possuir pelo menos um caso de teste específico.
- O alvo make test deverá compilar e executar automaticamente os testes.
- O resultado deverá indicar claramente aprovação ou falha.
- O grupo deverá demonstrar pelo menos um teste que falha inicialmente e passa após a correção do código.

### 3.6 Controle de versão com Git

- O Git deverá ser utilizado durante o desenvolvimento.
- O histórico deverá apresentar desenvolvimento incremental e coerente.
- Os commits deverão representar etapas relevantes da evolução do projeto.
- O projeto deverá utilizar pelo menos uma branch além da branch principal.
- Pelo menos uma funcionalidade deverá ser desenvolvida em uma branch separada e integrada por meio de merge.
- O repositório deverá possuir uma tag denominada v1.0.
- A tag v1.0 deverá corresponder a uma versão funcional, compilável e testável.
- Commits artificiais ou sem conteúdo relevante não serão considerados como evidência de desenvolvimento.

## 4. Requisitos Técnicos

- Linguagem C.
- GCC.
- Make.
- Git.
- Execução em ambiente Linux.
- Código organizado e documentado adequadamente.
- Separação entre biblioteca, arquivos de interface, testes e aplicação cliente.
- A compilação deverá utilizar opções apropriadas para as bibliotecas estática e dinâmica.

## 5. Estrutura sugerida do projeto

A estrutura abaixo é uma sugestão. O grupo poderá adotar outra organização, desde que a separação dos componentes seja clara.

```
projeto/
├── src/
│   ├── modulo1.c
│   ├── modulo2.c
│   ├── modulo3.c
│   └── modulo4.c
├── include/
│   ├── modulo1.h
│   ├── modulo2.h
│   ├── modulo3.h
│   └── modulo4.h
```

```
├── tests/
│   └── ...
├── app/
│   └── main.c
├── build/
├── lib/
├── bin/
├── Makefile
└── README.md
```

## 6. Instalação

O alvo install deverá instalar a biblioteca e os arquivos de interface em um prefixo definido pelo projeto. O Makefile deverá permitir alterar o prefixo para possibilitar instalações sem privilégios administrativos.

```
make install PREFIX=$HOME/.local
```

## 7. Relatório Técnico

O relatório deverá explicar como o projeto foi construído e deverá conter, no mínimo:

1. Descrição do problema e do domínio escolhido.
2. Arquitetura do software.
3. Organização e responsabilidade dos módulos.
4. Descrição da API pública.
5. Descrição da aplicação cliente.
6. Processo de compilação utilizando Makefile.
7. Estratégia de compilação incremental.
8. Geração e utilização da biblioteca estática.
9. Geração e utilização da biblioteca dinâmica.
10. Comparação entre ligação estática e dinâmica.
11. Estratégia e resultados dos testes.
12. Exemplo de teste que inicialmente falhou e posterior correção.
13. Estratégia de versionamento utilizando Git.
14. Descrição da branch, merge e tag v1.0.
15. Dificuldades encontradas e decisões de projeto.
16. Resultados obtidos e possíveis melhorias.

## 8. Demonstração

Durante a apresentação, o grupo deverá demonstrar:

17. Estrutura do projeto.
18. Interface pública da biblioteca.
19. Compilação utilizando make.
20. Geração da biblioteca estática.

21. Geração da biblioteca dinâmica.
22. Execução da aplicação utilizando a biblioteca estática.
23. Execução da aplicação utilizando a biblioteca dinâmica.
24. Execução dos testes utilizando make test.
25. Teste que inicialmente falha, seguido da correção e nova execução.
26. Compilação incremental após alteração de um módulo.
27. Execução de make clean.
28. Demonstração de uma branch.
29. Demonstração do merge.
30. Demonstração da tag v1.0.
31. Execução do processo de instalação.

## 9. Critérios de Avaliação

| Critério | Peso |
|---|---|
| Modularização, arquitetura e interface pública | 20% |
| Biblioteca estática | 10% |
| Biblioteca dinâmica e utilização pela aplicação | 10% |
| Makefile e compilação incremental | 20% |
| Testes automatizados | 15% |
| Uso adequado do Git | 10% |
| Relatório técnico e documentação | 10% |
| Organização e qualidade do código | 5% |

A avaliação considerará o funcionamento, a qualidade da solução e a capacidade do grupo de explicar as decisões adotadas.

## 10. Funcionalidades Extras

As funcionalidades abaixo não são obrigatórias, mas poderão ser utilizadas para ampliar o projeto:

- Depuração com GDB.
- Geração automática de documentação com Doxygen.
- Integração contínua utilizando GitHub Actions.
- Medição de cobertura de testes.
- Testes de desempenho.
- Exemplos adicionais de utilização da biblioteca.

## 11. Entrega

- Código fonte completo.
- Makefile.
- Repositório Git com histórico de desenvolvimento.
- README.md.
- Relatório técnico em PDF.
- Slides da apresentação.
- Vídeo de demonstração.
- Tag v1.0 no repositório.

O README deverá apresentar os pré-requisitos, a estrutura do projeto, a compilação, os alvos do Makefile, a execução, os testes e a instalação.

## 12. Sintaxe mínima

```
make

make clean

make run

make test

make install
```

O grupo poderá acrescentar outros alvos, desde que sua finalidade e utilização estejam documentadas no README.

## 13. Casos mínimos de teste

- Biblioteca com pelo menos 10 funções públicas.
- Pelo menos 4 módulos funcionalmente distintos.
- Aplicação cliente utilizando a API pública.
- Biblioteca estática gerada e utilizada.
- Biblioteca dinâmica gerada e utilizada.
- Testes automatizados para todas as funções públicas.
- Teste com entrada válida.
- Teste com valor limite ou entrada inválida, quando aplicável.
- Teste inicialmente com falha e posteriormente corrigido.
- Demonstração de compilação incremental.
- Demonstração de branch, merge e tag v1.0.
- Execução de make clean.
- Execução de make install.

## 14. Decisões deixadas para o grupo

- Domínio da biblioteca, sujeito à aprovação.
- Nome da biblioteca.
- Nome e organização dos módulos.
- Nomes e assinaturas das funções públicas.
- Estrutura interna dos diretórios.
- Estratégia de testes.
- Tratamento de erros.
- Organização dos artefatos de compilação.
- Prefixo de instalação.
- Organização do histórico Git.
- Funcionalidades adicionais.

Soluções diferentes serão aceitas quando atenderem aos requisitos e forem justificadas adequadamente.

## 15. Observações

- O trabalho deverá ser original.
- Durante a apresentação, qualquer integrante poderá ser questionado sobre qualquer parte do projeto.
- Cada integrante deverá ser capaz de explicar as decisões de implementação adotadas pelo grupo.
- O histórico Git deverá ser compatível com a evolução apresentada.
- Código cuja autoria ou funcionamento não possa ser explicado pelo grupo poderá ser desconsiderado.

## 16. Checklist antes da entrega

- ☐ A biblioteca possui pelo menos 10 funções públicas.
- ☐ As funções estão distribuídas em pelo menos 4 módulos.
- ☐ Cada módulo possui responsabilidade funcional própria.
- ☐ A API pública está definida nos arquivos .h.
- ☐ A aplicação cliente não inclui arquivos .c da biblioteca.
- ☐ A biblioteca estática foi gerada e utilizada.
- ☐ A biblioteca dinâmica foi gerada e utilizada.
- ☐ O Makefile possui all, clean, run, test e install.
- ☐ A compilação incremental foi implementada e demonstrada.
- ☐ make test executa automaticamente os testes.
- ☐ Todas as funções públicas possuem testes.
- ☐ Foram testadas situações normais e situações limite ou de erro, quando aplicáveis.
- ☐ Foi demonstrado um teste inicialmente com falha.
- ☐ O histórico Git apresenta evolução incremental.
- ☐ Existe branch além da branch principal.
- ☐ Foi realizado um merge.
- ☐ Existe a tag v1.0.
- ☐ O README está completo.
- ☐ O relatório está completo.
- ☐ Os slides estão completos.
- ☐ O vídeo foi produzido.

## 17. Resultado esperado

Ao final do trabalho, o grupo deverá entregar um projeto em C organizado como uma pequena biblioteca reutilizável, acompanhado de uma aplicação cliente, um sistema de compilação baseado em Make, testes automatizados e histórico de desenvolvimento em Git. A solução deverá ser reproduzível em ambiente Linux a partir das instruções apresentadas no README.